



Luxor university
Faculty of Computer and Information
Computer Science Department



**The Management Intelligent System
For Managing Individuals' Residence in Cities**

**Graduation Project
2024/2025**

**Under Supervision
Dr. Bassem Abd-El-Atty**

Submitted by

- | | |
|---------------------------|---------------------------|
| - Mohamed Badawy Sayed | - Mostafa Ahmed Hasan |
| - Mahmoud Ahmed Khalil | - Ahmed Nageh Abbas |
| - Nada Maaman Abdel Karim | - Basant Benyamen Barsoum |

Abstract

The Management Intelligent System For Managing Individuals' Residence in Cities, known as Semsary, is a web-based system designed with .NET, React, and Python (for machine learning integrations) to address the complex demands of short-term rental management in urban settings. Tailored for tenants, landlords, and families, Semsary streamlines the apartment rental experience by consolidating reliable listings, automating price assessments, and providing a secure booking environment.

Semsary tackles key challenges such as providing precise rental information, ensuring fair pricing, and connecting users to essential amenities. Through machine learning analysis, the platform recommends optimal rental prices and offers feedback to landlords on market standards, promoting transparency and helping renters make informed choices. This data-driven approach minimizes risk of fraud and ensures fair valuation, enabling users to find accommodations that align with their preferences and budget constraints.

Beyond a listing repository, Semsary's intuitive interface allows users to filter apartments by features like proximity to transportation and schools. Verified reviews foster informed decision-making, and location-based insights offer an enriched user experience. Secure, streamlined booking and payment options further simplify the rental process.

Semsary also features a dynamic administrative layer where managers handle advertising approvals, resolve disputes, and review tenant reports. Dedicated ID verification for both local and international users enhances trust and security on the platform. In cases of dissatisfaction, tenants can request refunds or rate properties, supporting accountability and enhancing service standards.

In conclusion, Semsary is positioned to drive positive change in urban rental management. By centralizing resources, integrating machine learning for equitable pricing, and enhancing user security and convenience, Semsary fosters a sustainable rental environment that aligns with the goals of sustainable urban growth and community well-being.

Table of contents

Abstract	- 2 -
Table of contents	- 3 -
Chapter 1: System Overview	- 4 -
1.1 Motivation	- 5 -
1.2 Problem Statement	- 5 -
1.3 Overview	- 6 -
1.4 Objectives and scope	- 6 -
1.5 Target Audience and Expected Impact	1
Chapter 2: Related Works	2
Chapter 3: Domain Analysis and Technique	4
3.1 Domain Analysis	5
3.1.1 Clients and users	5
3.1.2 The Environment	5
3.1.3 Tasks and actions currently being performed	5
3.2 Risks	5
3.3 Constrains	5
3.4 Project plan	5
3.5 Feasibility Study	5
3.6 Quality Assurance Plan	5
3.7 System Requirements	6
3.7.1 Functional Requirements	6
3.7.2 Non-Functional Requirements	7
3.8 Techniques and tools	9
3.8.1 Frontend Framework	9
3.8.2 Backend Development	11
3.8.3 Machine Learning Model	12
3.8.4 Third Party Services	13



Chapter 1

System Overview

1.1 Motivation

In today's world, people frequently move to different cities for various reasons, such as pursuing university studies, new work opportunities, vacations, and more. In addition to the increase in the number of expatriates to all governorates of Egypt in the recent period. Despite their varied purposes, one essential need unites them: finding a suitable place to live.

For many of these individuals, especially expatriates and students, the rental process can be challenging. Options for short-term, flexible, and furnished rentals are often limited, and securing a safe, well-priced residence in an unfamiliar city is not always straightforward.

Our project addresses this need with a platform designed to support sustainable cities and communities. By enabling homeowners to list available rentals and allowing tenants to compare and book our solution simplifies the rental process for everyone involved. In doing so, we aim to facilitate a more accessible, trustworthy, and sustainable rental market that empowers individuals and contributes to local community development.

Our motivation stems from the urgent need to transform our countries into sustainable cities and communities. While there are many factors to achieve this goal, developing the process of renting apartments to make it easier and smarter is one of the first steps to achieve this goal.

1.2 Problem Statement

Ineffectiveness of available methods of displaying available apartments, making it difficult for tenants to find the best match for their needs. Without clear information on prices, apartment contents, and nearby facilities, tenants struggle to make informed decisions about where to live.

Exploitation by brokers of both the landlord and the tenant, they get a commission from both of them in exchange for bringing the tenant to the landlord. They also often use misleading information to convince the tenant of the apartment.

There is a lack of effective methods to ensure that the tenant's expectations are met, which exposes him to the risk of receiving an apartment that does not conform to the agreed upon specifications or even that this apartment does not actually exist on the ground. This discrepancy between what was promised and what was delivered can lead to dissatisfaction and trust issues in the rental process.

Many tenants face challenges in obtaining real, transparent feedback about apartments, leaving them vulnerable to misinformation or deceptive practices. Without reliable reviews or insights, tenants cannot confidently evaluate the quality of an apartment before committing to a rental.

Landlords often struggle to reach a large and relevant audience of potential tenants, limiting their ability to fill available apartments efficiently. Without effective tools to market their

properties, landlords miss out on valuable opportunities to connect with suitable tenants who may be actively searching for housing options.

1.3 Overview

1.4 Objectives and scope

Our system, Semsary, is strategically designed to address key challenges in the short-term rental market, offering a streamlined, user-friendly, and data-driven system to facilitate apartment management and booking. By focusing on efficiency, security, and transparency, Semsary aims to revolutionize the urban rental experience for students, families, working individuals, and property owners alike.

Our primary objectives center on enhancing user experience, enabling data-backed rental pricing, simplifying booking processes, and ensuring informed decision-making. Through comprehensive apartment listings that detail essential features, amenities, and locations, Semsary empowers users to make well-informed choices, significantly reducing time spent on property searches. With advanced filtering options, users can effortlessly tailor their search to specific needs, including room count, rental type, and proximity to essential services such as transportation, schools, and markets.

Semsary also integrates machine learning capabilities to evaluate and recommend optimal rental prices based on input data, including room specifications, nearby services, and market trends. This approach provides landlords with pricing guidance aligned with market standards, while giving tenants confidence in fair and competitive pricing. The platform's data-driven pricing analysis supports transparency, minimizes the risk of fraud, and promotes financial fairness within the rental ecosystem.

In addition, the system enables seamless booking and secure payment options, which are essential for a smooth user experience. Property owners benefit from efficient request management, allowing them to review and respond to booking requests while ensuring exclusivity for each listing. A transparent request process also allows users to preview and cancel bookings within a specified time-frame, with funds held securely until completion or cancellation.

Semsary's comprehensive scope includes account creation, user interest customization, detailed transaction management, and a dynamic advertisement system, all contributing to an ecosystem that prioritizes sustainable urban living. By offering premium accounts, paid advertisements, and cancellation fees, Semsary not only enhances user experience but also ensures a sustainable revenue model.

1.5 Target Audience and Expected Impact

Our primary target audience includes Egyptian students, expatriate students, workers, Egyptian and expatriate families who are relocating to new cities for work, education, or personal reasons. These individuals often face challenges in finding short-term, flexible, and furnished rental accommodations that suit their needs. Additionally, landlords looking to rent out apartments or rooms to a wide range of tenants will also benefit from the platform. This includes homeowners in busy urban areas who require an easy and effective way to connect with potential tenants.

The expected impact of this platform is twofold: for tenants, it will make the process of finding reliable, affordable, and flexible housing options faster and more transparent, reducing the time and effort spent searching for suitable homes. For landlords, it will offer a more efficient way to connect with a wider pool of tenants, increasing the likelihood of renting out properties quickly. In the long term, the platform will contribute to building more sustainable communities by making housing access more equitable and transparent, promoting urban mobility, and supporting sustainable urban development.

Chapter 2

Related Works

Chapter 3

Domain Analysis and Technique

3 Domain Analysis and Technique

3.1 Domain Analysis

3.1.1 Clients and users

3.1.2 The Environment

3.1.3 Tasks and actions currently being performed

3.2 Risks

3.3 Constrains

3.4 Project plan

3.5 Feasibility Study

3.6 Quality Assurance Plan

3.7 System Requirements

3.7.1 Functional Requirements

Tenants:

1. Account Management:

- FR-1: The system should enable tenants to create an account and get verification via email.
- FR-2: The system should allow tenants to securely login using an email and password, and reset their login credentials if forgotten.
- FR-3: Tenants should be able to edit their profile information.

2. Search and Booking:

- FR-4: The system should allow tenants to search for properties and apply filters to find suitable listings.
- FR-5: Tenants should be able to send a booking request to a landlord if they have enough balance on the website.

3. Communication:

- FR-6: The system should enable tenants and landlords to communicate through an integrated chat feature.

4. Financial Transactions:

- FR-7: The system should allow tenants to deposit and withdraw money using credit cards or electronic wallets.
- FR-8: Tenants should be able to get back their money if they canceled the renting request during 4 days from date of request but after this period, the money will be transferred to landlord balance.
- FR-9: Tenants should be able to confirm their arriving to house to get their warranty money back.

5. Feedback and preview:

- FR-10: System should enable tenants to request for a house preview in case the house specifications do not match the advertisement.
- FR-11: Tenants should have the ability to give a rate of houses that they had already rented.

Landlord:

1. Account Management:

- FR-12: The system should allow landlords to create an account and get verification via SMS.
 - FR-13: The system should enable landlords to securely login using their phone number and password and reset their credentials if forgotten.
 - FR-14: Landlords should be able to update their profile information.
2. House Advertisement Management:
 - FR-15: The system should allow landlords to request an On-the-ground preview for their houses.
 - FR-16: The system should provide landlords access to all houses advertisements that have been inserted into their profile after preview.
 - FR-17: Landlords should be able to update specific details in their houses advertisements.
 - FR-18: Landlords should be able to publish their houses if they have enough balance and have the ability to delete them.
 3. Financial Transactions:
 - FR-19: The system should allow landlords to deposit and withdraw money using credit cards or electronic wallets.
 4. Rental Management:
 - FR-20: Landlords should be able to review, accept, or reject rental requests from tenants.
 - FR-21: The system should enable landlords to confirm tenants arrival requests to enable tenants get their warranty money back.

Customer Service:

1. Account Management:
 - FR-22: The system should allow customer service to securely login using an email and password, and reset their login credentials if forgotten.
 - FR-23: customer service should be able to edit their profile information.
2. Reviews Management:
 - FR-24: Customer service should be able to receive landlords requests for reviews and enter the house advertisement information.
 - FR-25: Customer service should be able to receive tenants' reviews requests.

3.7.2 Non-Functional Requirements

1. Performance Requirements:
 - NFR-1: The system should be able to handle up to 7,000 simultaneous user requests without performance degradation.
 - NFR-2: Page load time should not exceed 3 seconds under normal load conditions.
 - NFR-3: The platform should process payment transactions in under 5 seconds.
2. Scalability Requirements:

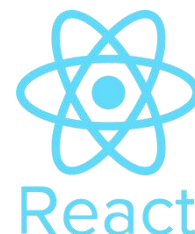
- NFR-4: The system should scale dynamically to accommodate an increase in users or data volume without performance degradation.
3. Availability Requirements:
 - NFR-5: The platform should have an uptime of at least 98% annually.
 - NFR-6: The system should provide redundancy mechanisms to minimize downtime during maintenance or unexpected failures.
 4. Security Requirements:
 - NFR-7: All user data must be encrypted during transmission using HTTPS and TLS protocols.
 - NFR-8: Passwords should be hashed and stored securely using an industry-standard hashing algorithm.
 - NFR-9: The system must be safeguarded against common web vulnerabilities like XSS, SQL injection, etc.
 5. Usability Requirements:
 - NFR-10: The user interface should follow accessibility standards and be user-friendly.
 6. Reliability Requirements:
 - NFR-11: The system should recover from critical failures within 10 minutes.

3.8 Techniques and tools

3.8.1 Frontend Framework

1. React JS:

React.js was selected as the core framework for developing the frontend of the apartment rental platform due to its modular and dynamic nature. Key features used include:



Component-Based Architecture:

Built reusable components such as property cards, search bars, filters, and user authentication modals. Ensured consistency and reduced redundancy by organizing components logically within feature-based folders.

React Hooks:

Used `useState` to manage states like filter selections, form inputs, and user authentication. Implemented `useEffect` for handling side effects, such as fetching property data from Firebase in real-time and monitoring changes.

React Router:

Integrated React Router to enable navigation across pages (e.g., Home, Apartment Listings, User Profile, and About Us) without full page reloads, creating a seamless user experience.

2. Styling Framework:



Tailwind CSS:

Tailwind CSS was used for its efficiency in designing modern, responsive, and visually appealing interfaces tailored for real estate listings.

Custom Themes:

Configured Tailwind's theme in `tailwind.config.js` to reflect a professional color palette, such as warm neutrals and blues, representing trust and reliability in real estate.

Responsive Design:

Ensured the website was mobile-friendly by designing layouts with responsive utility classes to accommodate all screen sizes, from smartphones to large monitors.

3. State Management

To manage data flow and application state:

React Context API:

Used for sharing global states like active filters (e.g., price range, location, and amenities) and user login status across the application without prop drilling.



Local State Management: Managed component-specific states for dynamic features like search queries and form validations.

Redux

Redux was used to handle the global state of the apartment rental platform efficiently. Key features include:

Centralized State: Managed filters, apartment listings, user authentication, and reviews in a single global store.

Actions and Reducers: Defined reusable actions (e.g., FILTER_APARTMENTS, ADD_REVIEW) and reducers to update the state predictably.

Middleware: Used redux-thunk for asynchronous tasks like fetching apartment data and submitting reviews.

React Integration: Connected Redux to React using react-redux, utilizing useSelector for accessing state and useDispatch for dispatching actions.

Improved Performance: Cached data like apartment listings to minimize API calls and enhance responsiveness.

4. Build Tools: Vite and npm

Vite: Used for fast development with Hot Module Replacement (HMR) and optimized production builds. Custom configurations in vite.config.js supported Tailwind CSS and Firebase integration.



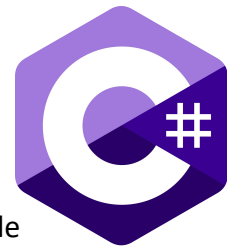
npm: Managed dependencies (e.g., React, Redux, Firebase) and provided scripts for development (npm run dev), production build (npm run build), and preview (npm run preview).



3.8.2 Backend Development

1.C#

C# was chosen as the programming language for backend development due to its versatility and strong integration with .NET. Key features include:



Object-Oriented Programming: Enabled modular, reusable, and maintainable code, essential for implementing business logic for user management, apartment listings, and reviews.

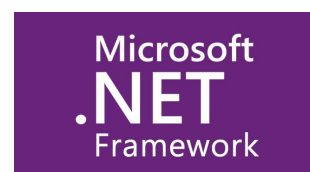
Asynchronous Programming: Used `async/await` for handling asynchronous tasks like database queries and API calls, improving server responsiveness.

Robust Error Handling: Implemented try-catch blocks and custom exception handling to ensure stability and provide detailed error messages for debugging.

Integration Support: C#'s rich library ecosystem facilitated seamless integration with Firebase for image storage and third-party tools like Google Analytics.

2.NET Framework

ASP.NET Core, a modern and high-performance web framework, powered the backend with the following features:



RESTful API Development: Designed endpoints for CRUD operations on apartments, user accounts, and reviews. Supported HTTP methods like GET (retrieve apartments), POST (add reviews), PUT (update user info), and DELETE (remove listings).

Middleware: Used custom middleware for logging, error handling, and request validation. Implemented authentication middleware to validate tokens and secure API access.

Dependency Injection:

Integrated DI to manage services like database contexts and external API clients, enhancing code modularity and testability.

Authentication and Authorization:

Implemented JWT (JSON Web Token) for user authentication and role-based authorization to differentiate access for normal users, premium users, and admins.

1. SQL Server

SQL Server served as the relational database for managing structured data in the application. Key aspects include:



Database Design:

Tables were designed with proper normalization to reduce redundancy and improve data integrity.

Users: Stored user details, authentication information, and roles.

Apartments: Included details like location, price, amenities, and image URLs.

Reviews: Managed user feedback with fields for ratings, comments, and timestamps.

Stored Procedures:

Created stored procedures for complex queries like fetching apartments with filters applied and calculating average ratings.

Improved performance by reducing the load on the application layer and optimizing query execution.

Indexes:

Added indexes on frequently queried fields (e.g., apartment location, price) to speed up search operations.

Transactions:

Used transactions to ensure consistency during critical operations, such as booking apartments or submitting reviews, to prevent partial updates.

3.8.3 Machine Learning Model

1. Python

Python served as the primary programming language for implementing the recommendation system due to its:



Rich Ecosystem: Extensive libraries and tools for data manipulation, visualization, and machine learning.

Ease of Use: Simple syntax that accelerates development and experimentation.

Community Support: Access to a vast range of resources, tutorials, and pre-built models.

2. TensorFlow

TensorFlow, an open-source machine learning framework, was used for building and training the recommendation model.



Key Features Utilized:

Deep Learning Support: Designed a neural network architecture optimized for collaborative filtering or content-based recommendations.

Ease of Deployment: TensorFlow's model export capabilities allowed easy integration of the trained model with the backend.

TensorFlow Recommenders (TFRS): A specialized library within TensorFlow, TFRS provided prebuilt tools for creating recommendation systems, reducing development time.

Advantages:

Scalability: Handled large datasets of user reviews, apartment details, and preferences efficiently.

Performance: TensorFlow's GPU support accelerated training times for large-scale models.

Customizability: Allowed fine-tuning of the model to cater to unique aspects of the rental platform, such as prioritizing recent reviews or higher-rated apartments.

3.8.4 Third Party Services

1. Firebase Storage

Firebase Storage was utilized for storing and managing user-uploaded images, such as apartment photos. Its integration simplified handling large files and ensured reliable, scalable storage. Key features include:

Seamless Integration:

Integrated with the Firebase SDK to easily upload, retrieve, and manage image files directly from the application.

Scalability:

Supported storing high-resolution images without impacting app performance, ensuring the platform can handle increasing user activity.

Security Rules:

Configured Firebase security rules to control access, ensuring only authenticated users can upload images and restricting downloads as per user roles.



Cloud Storage
for Firebase

Real-Time URL Generation:

Generated secure download URLs after each upload, which were stored in the database and used to display images on the platform.

Cost-Effectiveness:

Benefited from Firebase's pay-as-you-go pricing, making it a cost-efficient choice for storage needs.

2. SendGrid:

Email Delivery Service

SendGrid is a cloud-based email delivery platform that provides businesses with the tools to send transactional and marketing emails at scale. It offers a range of APIs and SMTP services for email delivery, along with analytics to track email performance, ensuring high deliverability rates and reliable service. SendGrid is popular among developers for its ease of integration with both small applications and large-scale enterprise systems.



Email API: Allows developers to send email messages from their applications, with options for both transactional and marketing emails.

SMTP Relay: Provides a reliable service for sending emails using the SMTP protocol, making it easy to integrate into existing systems.

Deliverability Tools: Includes tools to track email delivery, open rates, click rates, and more, to optimize performance.

Templates and Personalization: Users can create and send customized email templates for better engagement with recipients.

Scalability: Supports both small and large-scale email sending needs, suitable for businesses of any size.

Security: Offers features like email authentication and encryption to protect against phishing and unauthorized access.

Use Cases:

Sending transactional emails (e.g., order confirmations, password resets).

Running marketing campaigns with advanced email personalization.

Integrating email services with web applications via API.

3. PayMob: Payment Gateway



PayMob is a leading payment gateway in Egypt, designed to facilitate secure online transactions for businesses and consumers. It provides an API for processing payments, enabling businesses to accept payments via credit/debit cards, mobile wallets, and other local payment methods. PayMob is widely used by e-commerce businesses, subscription services, and retail merchants to offer a seamless and secure payment experience.

Key Features:

Payment Gateway API: Allows businesses to process payments from customers via a wide range of local and international payment methods.

Mobile Wallet Integration: Supports popular mobile wallets, including Fawry, Vodafone Cash, and others for easy mobile-based payments.

Recurring Payments: Enables subscription-based services to collect payments on a regular basis.

Fraud Detection: Provides advanced security features, including fraud prevention tools, to protect businesses from fraudulent transactions.

Invoicing: Facilitates generating and managing invoices directly from the PayMob platform.

Local Integration: Tailored specifically for the Egyptian market, with support for local banks and payment methods.

Use Cases:

Enabling e-commerce sites to accept payments securely.

Providing subscription-based businesses with recurring billing solutions.

Integrating mobile wallet payments for greater convenience.

4. Twilio: SMS Messaging API

Overview:

Twilio provides a powerful SMS API for businesses to send and receive text messages globally. It allows seamless integration of SMS capabilities into web and mobile applications, enabling businesses to automate notifications, alerts, reminders, and two-way messaging. With robust scalability and reliable delivery, Twilio ensures effective communication for organizations of all sizes.



Key Features:

Send SMS:

Quickly send text messages to customers worldwide through a simple API.

Receive SMS:

Enable two-way communication by receiving replies directly in your application.

Phone Number Management:

Purchase, configure, and manage virtual phone numbers to customize messaging.

Global Reach:

Deliver SMS to over 180 countries with high reliability and fast delivery rates.

Programmable Messaging:

Personalize messages with templates and variables for tailored communication.

Delivery Status Tracking:

Monitor the status of sent messages with detailed delivery reports.

Scalability:

Supports large-scale messaging for campaigns, promotions, or transactional alerts.

Use Cases:

Sending transactional messages like OTPs, confirmations, and alerts.

Running SMS-based marketing campaigns.

Automating customer notifications, reminders, and updates.

5. Google Analytics

Google Analytics was integrated into the project to track and analyze user interactions, providing valuable insights for improving the platform.



Key Features:

Real-Time Monitoring:

Tracked active users, pages visited, and interactions as they occurred.

User Behavior Analysis:

Collected data on how users navigated the site, including frequently visited pages, filters used, and time spent on listings.

Traffic Sources:

Identified where users originated from (e.g., search engines, social media, or referrals), helping optimize marketing strategies.

Event Tracking: Monitored specific actions, such as apartment searches, clicks on "Contact Owner," and image uploads.

Helped understand user preferences and behavior, enabling data-driven improvements to the platform.

Monitored conversion rates, such as users signing up or submitting apartment reviews.

6. Version Control: Git and GitHub

Git: Used locally for tracking code changes, managing feature branches and enabling easy rollbacks.



GitHub: Hosted the repository for collaboration, pull requests, and code reviews, while serving as a secure backup.

